Kurve RSC

Multi-Cutoff Relational Signal Compression, Downstream Learning, and Kurve RSC Feature Families

Kurve RSC Technical Report

August 24, 2026

Contents

1 Abstract

Kurve RSC is a framework for **relational signal compression**: the conversion of a time-varying, multi-table database into leakage-safe, model-ready rows at a chosen prediction grain. Rather than flattening the database once, Kurve RSC instantiates a relational compute graph repeatedly at a configurable sequence of cutoffs. Every instantiation emits one dataframe containing the entities, features, cutoff, and task label for that point in time. The cutoff-specific frames form one logical training matrix that can fit, condition, or fine-tune any compatible downstream learner. This includes conventional gradient-boosted trees and neural models as well as tabular foundation models such as TabPFN.

Two independent top-level controls define the scale of the representation. The **frame count** K determines how many historical cutoff views are generated. The **relational depth** d determines how many foreign-key hops can contribute signal to each view. Feature-family selection then determines what information survives each many-to-one reduction. These controls expose a clear trade-off between relational coverage, temporal coverage, feature width, row count, and computation.

Kurve RSC uses the open-source GraphReduce project as its relational execution engine. GraphReduce represents tables as nodes, keys as edges, and filtering, annotation, aggregation, and joining as an executable compute graph. Kurve RSC supplies the benchmark-facing orchestration around that engine: schema adaptation, cutoff scheduling, repeated frame generation, feature alignment, disk-backed frame storage, downstream learner adaptation, official evaluation, and prediction-file validation.

2 Executive summary

The central idea is easier to understand as a loop than as one large join:

```
for cutoff c in selected_cutoffs:                  # K is configurable
    graph = instantiate(database, root, depth=d)  # d is configurable
    graph.filter_to_information_visible_at(c)
    graph.reduce_relations_to_the_root_grain()
    frame[c] = graph.entity_dataframe() + task_labels[c]

training_matrix = align_and_union(frame[c] for c in selected_cutoffs)
learner = fit_condition_or_finetune(training_matrix, validation_frame)
```

Each frame answers a precise historical question: *what could the system have known about this entity at this cutoff?* Repeating that question at several cutoffs turns a relational database into a longitudinal tabular training set. The resulting matrix contains repeated entity keys when the same entity is eligible at more than one cutoff; those rows are distinct observations because their visible histories and future label windows differ.

Kurve RSC separates four responsibilities:

Layer	Responsibility
Task specification	Official entity keys, timestamps, labels, temporal splits, and metrics
Kurve RSC	Cutoff schedule, depth, feature policy, frame lifecycle, training, and validation
GraphReduce	Point-in-time graph execution, bottom-up reduction, and feature propagation
Downstream learner	Mapping the unified entity representation to a prediction

This separation is deliberate. RelBench defines the prediction problem; GraphReduce executes relational algebra; Kurve RSC defines the repeated representation-learning workflow; and the emitted dataframe is a learner-agnostic interface. CatBoost is the default conventional learner, but the same frame contract supports linear models, neural tabular models, AutoML systems, and—most interestingly—pretrained tabular foundation models such as TabPFN. Kurve RSC's main contribution is the relational and temporal representation presented to whichever learner follows it.

The rest of this report proceeds from the system-level view to the detailed feature program. The first sections explain multi-cutoff frame generation, graph depth, point-in-time correctness, and downstream learning; the **Kurve RSC Feature Families** and their interactions follow. Later sections discuss cost, prior work, reproducibility, limitations, and GraphReduce's role.

3 Kurve RSC at a glance

3.1 Inputs and output

Kurve RSC begins with:

- a relational database $\mathcal{D}$ whose tables expose primary and foreign keys;
- a prediction task $\mathcal{T}$ with an entity table, entity key, timestamp, target, split schedule, and evaluator;
- a cutoff sequence $C = (c_1, c_2, \dots, c_K)$;
- a graph-depth budget d;
- a feature-family policy $\mathcal{A}$; and
- a finite estimator configuration set $\mathcal{H}$.

For every cutoff c_i, the framework emits an entity-grain frame

$$F_i = \left[e,\ c_i,\ \phi_1(e, c_i), \dots, \phi_p(e, c_i),\ y(e, c_i)\right],$$

where e is an official task entity, each ϕ_j is computed only from information visible at c_i, and $y(e, c_i)$ is supplied by the task's future label window. The logical training relation is the row union

$$F_{\text{train}} = F_1 \oplus F_2 \oplus \cdots \oplus F_K.$$

The symbol $\oplus$ means schema-aligned row concatenation. It does not mean that every frame must be held in memory simultaneously; Kurve RSC can persist the parts and stream them during training.

3.2 The complete lifecycle

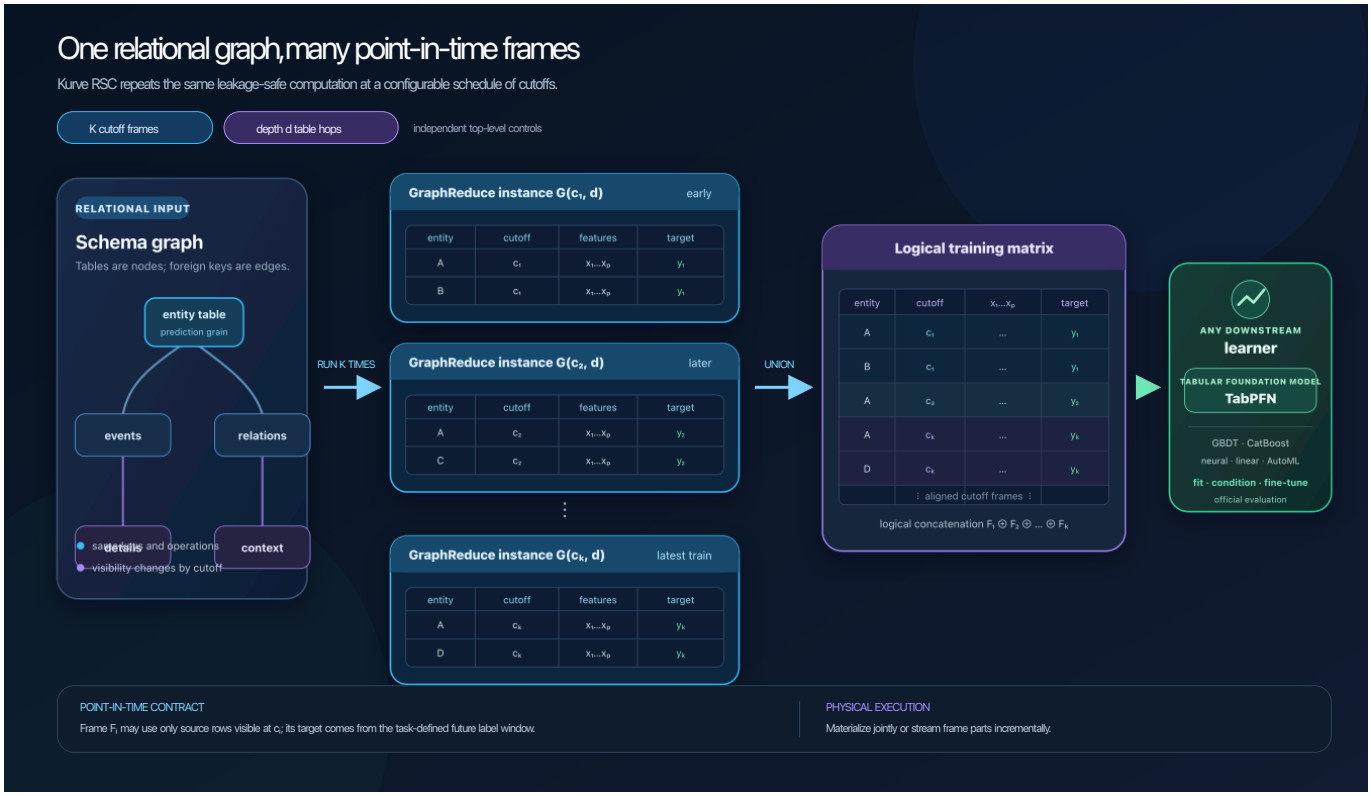

Figure 1: Kurve RSC repeats a relational compute graph at K cutoffs, emits one table per cutoff, and exposes their unified representation to any downstream learner, with tabular foundation models such as TabPFN highlighted.

The lifecycle has seven stages:

1. **Read the task contract.** Load the official entity keys, timestamps, labels, train/validation/test partitions, and evaluator.

2. **Construct the relational graph.** Represent source tables as graph nodes and primary–foreign key relations as edges. Select the root entity and the depth budget.
3. **Instantiate at a cutoff.** Bind a fresh graph execution to c_i, the historical compute horizon, and the task entities eligible at that cutoff.
4. **Compress relational history.** Filter future information, compute node features, reduce one-to-many relations, and propagate the summaries toward the root.
5. **Emit one entity frame.** Join the root representation to the matching task rows and persist the result as F_i.
6. **Align the frames.** Freeze a compatible feature order and create the logical row union across selected training cutoffs.
7. **Adapt and evaluate a downstream learner.** Fit a conventional estimator, condition a pretrained learner in context, or fine-tune a compatible model; then score with the official evaluator. Test labels do not participate in selection.

3.3 Two independent scale controls

The most important architectural distinction is between **how often** the graph is evaluated and **how far** signal travels inside each evaluation.

Control	Symbol	Changes	Primarily grows
Cutoff-frame count	K	Number of historical graph instantiations	Training rows and graph executions
Relational depth	d	Foreign-key distance from the prediction root	Included tables, feature width, and reduction work

Increasing K does not include a deeper table. Increasing d does not create another historical observation. A run can therefore be temporally broad but relationally shallow, relationally broad but temporally sparse, broad on both axes, or deliberately small on both.

At the architecture level, K is any positive integer supported by the task's valid timestamp schedule. In the benchmark harness the selected cutoff list is the top-level realization of K: a task may use the complete official schedule, one canonical cutoff, or a reproducibly selected bounded subset. Likewise, d is a nonnegative graph parameter, realized by GraphReduce's directional auto-feature hop settings. Both should be reported with results.

4 Multi-cutoff frame generation

4.1 Why instantiate the graph more than once?

A single snapshot teaches the estimator about one historical state. A cutoff sequence teaches it how similar entities look at several stages of their history. For a user task, the same user may have little activity at c_1, a bursty recent history at c_2, and a mature history at c_K. For a clinical study, each cutoff may expose a different set of reported events, facilities, or outcomes. The entity key can repeat, but the row represents a new point-in-time prediction opportunity.

The graph topology and feature policy ordinarily remain fixed across the loop. What changes is the cutoff-dependent visibility of source rows, the eligible task entities, and the associated future label window. This repeated instantiation is what turns relational feature synthesis into a training-set generator rather than a one-time reporting query.

4.2 One frame per cutoff

Suppose two entities are eligible at three cutoffs. Kurve RSC builds the frames independently:

Frame F_1 at cutoff c_1

entity	cutoff	lifetime events	30-day events	amount sum	target
A	c_1	3	2	45	0
B	c_1	1	1	12	1

Frame F_2 at cutoff c_2

entity	cutoff	lifetime events	30-day events	amount sum	target
A	c_2	8	4	131	1
C	c_2	2	2	28	0

Frame F_K at the latest selected training cutoff

entity	cutoff	lifetime events	30-day events	amount sum	target
A	c_K	14	3	247	0
D	c_K	5	5	91	1

After schema alignment, the logical training matrix is:

entity	cutoff	lifetime events	30-day events	amount sum	target
A	c_1	3	2	45	0
B	c_1	1	1	12	1
A	c_2	8	4	131	1
C	c_2	2	2	28	0
A	c_K	14	3	247	0
D	c_K	5	5	91	1

This is a row union, not a key-based merge between cutoffs. Joining F_1 to F_2 horizontally would put features from different prediction times on the same row and would destroy the intended training semantics.

4.3 The cutoff schedule is a top-level parameter

Let C_{valid} be the task's valid training schedule. Kurve RSC selects an ordered subset $C \subseteq C_{\text{valid}}$ and sets $K = |C|$. Useful policies include:

- **latest only:** $K = 1$, useful for fast diagnostics or memory-constrained experiments;
- **full schedule:** every valid official training cutoff;
- **bounded schedule:** a deterministic subset, normally retaining the latest cutoff and spreading the remainder over history; and
- **domain schedule:** cutoffs chosen at meaningful task intervals, provided every selected point obeys the official task contract.

Frame count and construction concurrency are separate. `K=50` means fifty historical frames. `training_frame_workers=15` means at most fifteen frames are built concurrently; it does not reduce K to fifteen.

4.4 Independent frames enable parallel construction

Once the database sources are registered, F_i depends on c_i but not on the output of F_{i-1}. The computation is therefore embarrassingly parallel at the frame level. Kurve RSC can give each worker its own DuckDB cursor and yield completed frames in deterministic schedule order.

Parallelism changes wall-clock time and peak resource usage, not feature semantics. A reproducible run records both K and worker count because two runs may use identical training rows while having very different execution profiles.

4.5 Logical union, physical streaming

The notation $F_{\text{train}} = \oplus_i F_i$ describes the logical dataset. Kurve RSC supports two physical interpretations:

Mode	Physical behavior	Strength	Cost
Joint	Materialize aligned parts into one in-memory matrix before fitting	Conventional global fit and early stopping	Higher peak memory

Mode	Physical behavior	Strength	Cost
Incremental	Read one persisted frame or batch at a time and continue the same model	Bounded memory	Training order and per-frame iteration allocation matter

The frame store keeps small runs in memory and spills larger runs to numbered Parquet parts. This separates representation generation from model memory: all selected frames can participate even when their combined dataframe does not fit comfortably in RAM.

5 Relational depth and signal coverage

5.1 Database metadata becomes a compute graph

Kurve RSC consumes schema metadata rather than beginning with a pre-flattened table. For each source relation it needs a stable name, primary key, optional time key, and eligible columns. Foreign-key declarations connect child rows to their parent entities. The task's entity table becomes the root grain.

GraphReduce [1] executes this representation as a graph of table nodes and join edges. Kurve RSC uses bottom-up reduction so a one-to-many child is grouped to one row per parent key before joining upward. This preserves parent cardinality and avoids the uncontrolled fan-out of a raw multi-table join.

5.2 Depth is a top-level coverage parameter

Define the included table set at depth d as

$$V_d = \{v \in V : \mathrm{dist}(v, r) \leq d\},$$

where r is the prediction entity table and distance is measured along eligible primary–foreign key edges. The directional GraphReduce hop controls refine which inward or outward paths can propagate features.

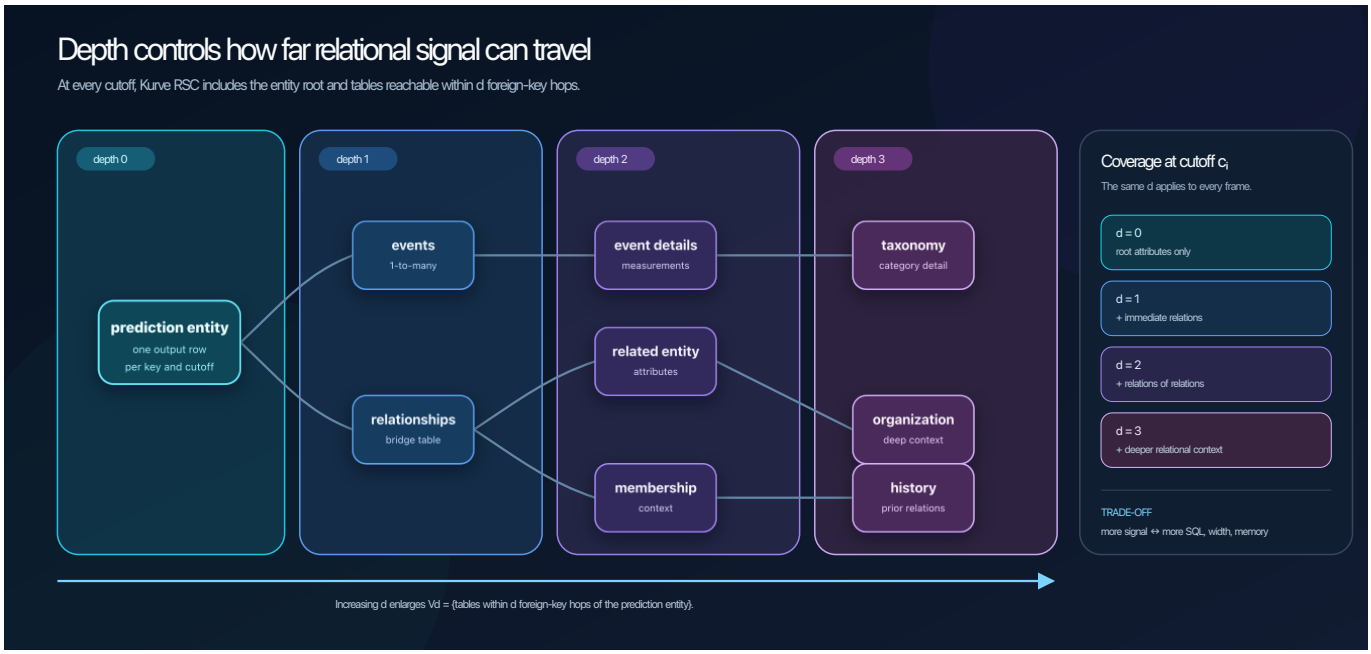

Figure 2: Increasing relational depth admits tables at greater foreign-key distance from the prediction entity.

At each cutoff:

- $d = 0$ uses only attributes already present at the prediction entity grain;
- $d = 1$ adds immediate events and bridge relations;

- $d = 2$ adds relations of those relations, such as referenced entity attributes or event details; and
- larger d admits progressively deeper relational context.

The depth is arbitrary in the sense that it is a top-level nonnegative parameter, not a feature formula tied to a task. Its effective maximum is the reachable schema diameter and its practical maximum is constrained by compute budget.

5.3 Signal propagates by reduction

Consider a three-hop path:

```
prediction entity <- transactions <- line items -> products
       depth 0           depth 1        depth 2       depth 3
```

At depth three, product attributes can be summarized at the line-item grain, line-item summaries can be reduced to each transaction, and transaction summaries can be reduced to the prediction entity. Each stage changes the grain before the next join. The final root row may therefore contain both direct transaction statistics and summaries derived from deeper product context.

A feature family is evaluated at a node and reduction hop; it is not one fixed global column list. The same `temporal` policy can create different columns on an event table and a transaction table because their schemas, dates, and reduction keys differ.

5.4 Depth, cycles, and totality

"Use all available relational signal" must always be qualified by the chosen depth and edge policy. Within the configured subgraph, the intended behavior is to expose every eligible table and column to generic reduction. A shallower depth deliberately excludes farther signal.

Real schemas may contain cycles or multiple paths to the same table. Blindly unrolling every path can repeat a relation indefinitely or multiply equivalent features. A production graph builder therefore needs a deterministic cycle policy: for example, a cycle-free traversal, explicit edge orientation, or a bounded path expansion with stable namespaces. The selected policy must be reported because "all tables" and "all paths" are not the same claim.

5.5 Coverage and cost

Increasing d can increase:

- the number of scanned tables and edges;
- SQL plan size and intermediate materializations;
- feature width after each reduction;
- categorical expansion and windowed aggregates; and
- peak memory, especially when already-reduced columns are re-aggregated at another hop.

The cost can grow faster than linearly when width compounds across hops. Depth should therefore be treated as an experimental axis and resource budget, not as an automatic instruction to choose the schema maximum.

6 Point-in-time correctness

6.1 The visibility contract

For a prediction cutoff c_i, every feature must be computable using only source information available at or before the task's boundary convention. The task label belongs to a future window and is joined only after feature construction. In symbols, a feature row should satisfy

$$\phi(e, c_i) = f(\{x : t(x) < c_i\}),$$

or the task's documented inclusive equivalent. A SQL engine's strict versus inclusive comparison must be reconciled explicitly at the adapter boundary.

6.2 Cutoff, compute horizon, and label horizon

Three time concepts should not be conflated:

Concept	Purpose
Cutoff c_i	Point at which the prediction is made
Compute horizon	How far backward source rows may contribute
Label horizon	Future interval used by the task to construct $y(e, c_i)$

A 365-day feature window cannot recover rows if the compute horizon admits only 90 days. Conversely, extending the compute horizon must never cross the cutoff into the label window.

6.3 Time-aware nodes

Every time-varying relation should declare the timestamp used for filtering. GraphReduce can then apply the cutoff during aggregation and calculate recency, window counts, and family-specific temporal features relative to the graph's reference time. Static dimension tables need no date filter but still must be joined through a valid historical schema relationship.

6.4 Split discipline

Kurve RSC keeps these roles separate:

- training cutoffs generate labeled fitting frames;
- validation cutoffs select model settings and may determine early stopping;
- test rows receive predictions only after the representation and model policy are fixed; and
- official task evaluators compute the reported metric.

Target-derived decisions must use training and validation only. The test split must not select feature families, depth, frame count, category values, model configuration, clipping rules, or blends.

7 From frames to downstream learning

7.1 Learner interoperability

Kurve RSC terminates in an ordinary dataframe contract rather than a model-specific hidden representation. Once the cutoff frames are aligned, any learner that accepts tabular rows can sit downstream:

- gradient-boosted trees such as CatBoost, XGBoost, or LightGBM;
- linear, generalized linear, neural-tabular, and AutoML systems;
- in-context or fine-tuned tabular foundation models; and
- task-specific learners that preserve the same split and prediction-key contract.

This interoperability is architectural, not a claim that every possible library already has a bundled adapter. The current benchmark harness provides CatBoost as its conventional default and TabPFN as its tabular-foundation-model backend. Additional learners need only consume the frozen feature matrix and return predictions in official task-row order.

Tabular foundation models are especially interesting here. Kurve RSC can act as a relational context compiler: it turns a depth-bounded, time-censored database into the rows and columns on which a pretrained tabular model reasons. TabPFN can condition on the generated training matrix in context and supports fine-tuning [6]; newer Prior Labs systems broaden the supported scale and structured-data settings [7]. The relational representation and downstream statistical prior remain separable, so either side can improve without redesigning the other.

7.2 Feature-schema alignment

Different cutoffs can expose different sampled categories or sparse columns. Before fitting, Kurve RSC selects a stable model-facing schema and order. Training, validation, and test frames are reindexed to that layout. Missing numeric values receive a consistent numeric representation; categorical columns receive a stable missing token and categorical index layout.

Feature alignment protects against two failure modes: allowing a late cutoff to silently change the model's column meaning, and passing a validation/test column that never existed during training.

7.3 Training and adaptation protocol

For conventional supervised learners, Kurve RSC can compare a finite set of ordinary configurations. In the reference CatBoost path, classification candidates are compared by validation AUROC and regression candidates by validation error. The selected configuration is then used for validation/test prediction under the frozen feature policy.

This report uses **fine-tuned** in the benchmark sense: the downstream model's ordinary hyperparameters are selected on validation data and its parameters are adapted to the generated relational frames. For a foundation model, the more precise operation may be in-context conditioning, inference-time configuration, or gradient-based fine-tuning. Those modes should be named explicitly rather than collapsed into one ambiguous use of "training."

CatBoost [4] is the default because it handles mixed numerical and categorical tables while providing a strong conventional tabular baseline. TabPFN [6] is a first-class alternative because the unified Kurve RSC frame is already the tabular context it expects. Foundation-model context limits and row/column budgets may favor a smaller K, a smaller d, or bounded feature families; that resource choice does not change the point-in-time frame contract.

7.4 Incremental versus all-at-once fitting

With joint fitting, all selected frame parts are materialized into F_{train} and one model fit sees the complete matrix at once. With incremental fitting, the same logical frames are replayed in cutoff order and CatBoost continues from the prior model. Iterations are allocated across the available batches so the total configuration budget remains controlled.

The two modes optimize different operational constraints. Joint fitting is conceptually simplest and can use global early stopping. Incremental fitting supports much larger logical training sets with bounded peak memory. The mode is part of the experimental configuration and should be reported. In-context foundation models ordinarily consume a jointly materialized context or their own chunked/context-management mechanism rather than CatBoost's continuation semantics.

7.5 Evaluation and output integrity

Classification tasks use the official task metric, commonly AUROC. Regression tasks use the official error metric and Kurve RSC may additionally report normalized mean absolute error,

$$\text{NMAE} = \frac{\text{MAE}(y, \hat{y})}{\text{std}(y_{\text{train}})}.$$

The normalization denominator comes from training targets only. Submission generation joins predictions back to untouched official test keys and checks exact key coverage before writing a file.

8 Kurve RSC Feature Families

Feature families specify **what information survives relational compression**. Kurve RSC brands and composes these reusable policies as part of its representation layer; GraphReduce's SQL auto-feature planner executes the corresponding node-level operations [1]. The families operate on schemas, types, timestamps, annotations, and reduction keys rather than on a hard-coded feature table for one prediction target.

8.1 Family overview

The SQL representation program recognizes seven Kurve RSC feature families:

Family	The question it answers	Typical output
`base`	How much, how many, and how recently?	Type-aware aggregates, category/text summaries, recency, window counts
`semantic`	What domain concept does this row represent?	Caller-defined predicate or value annotations
`conditional`	What kind of activity occurred in each window?	Conditional count, share, presence, and change
`temporal`	How did numeric magnitude vary by lookback window?	Windowed sum, average, minimum, and maximum
`sequence`	Was activity steady, concentrated, or bursty?	Rate, recent share, burst ratio, active span
`episode`	How many rows versus distinct real-world records?	Row counts and distinct-primary-key counts
`context`	How does a row compare with its peers?	Peer-group size and difference from peer mean

> **Automation boundary.** `semantic` is **custom, declarative, and opt-in**. Kurve RSC does not infer domain concepts such as success, severity, or value. The caller must supply each semantic predicate or value expression. Enabling the family without those expressions produces no domain-semantic features.

The families differ in how much caller knowledge they require:

Automation class	Families	What the framework supplies automatically
Schema/type driven	`base`, `temporal`, `sequence`, `episode`	Operations are synthesized from eligible columns, types, timestamps, and keys after the family is enabled
Hybrid	`conditional`	Conditions can come from sampled categorical values, while explicit semantic predicates take priority
Caller guided	`semantic`, `context`	`semantic` requires expressions; `context` requires a defensible peer-group key

`auto_annotate_features=True` is a separate heuristic option. It can create bounded generic annotations from column shape and sampled values, but it does not discover domain meaning and does not turn `semantic` into an automated semantic-understanding system.

The practical baseline is `base`. Add families where their signal has a clear interpretation: `temporal` for measurements, `sequence` for cadence, `conditional` for status or category mix, `episode` for duplicate-prone joins, `semantic` for stable domain rules, and `context` for a defensible peer group.

> **Implementation scope.** The seven named families described here are implemented by GraphReduce's SQL auto-feature planner and executed through the SQL graph transformation path. GraphReduce also supports other compute backends, but their automatic aggregation paths need not implement the same seven-family program.

8.2 The family mental model: annotate, reduce, then join

Suppose the desired output is one row per user and each user has many events:

```
users (parent grain)          events (child grain)
+---------+                  +----------+---------+--------+--------+
| user_id |  1 <------------ | event_id | user_id | status | amount |
+---------+        many      +----------+---------+--------+--------+
```

With a reducing edge, the child relation is processed before it is joined to the parent:

```
raw child rows
    -> point-in-time filtering
    -> row annotations (`semantic` and `context`)
    -> grouped feature calculation by the edge key
    -> one reduced row per parent key
    -> left join into the parent
```

This ordering explains two ideas:

1. A family describes operations at a node and graph hop, not a fixed list of global model columns.
2. `semantic` and `context` enrich child rows first. Ordinary reduction then summarizes those enriched values to the parent grain.

The graph must enable automatic features and execute the SQL transformation path. Kurve RSC repeats that same family program at every selected cutoff.

8.3 Running family example

The examples below use a cutoff of **2024-01-10** and lookback periods of 1, 7, and 30 days. All rows belong to user 1.

event_id	event_time	status	amount
101	2024-01-01	invited	10
102	2024-01-05	paid	20
103	2024-01-09	invited	30

At the cutoff, the 1-day window contains one event, the 7-day window contains two, and the 30-day window contains all three. Rows after the reference time do not contribute.

8.4 `base`: schema-aware rollups

8.4.1 What it does

`base` is the broad baseline. The engine samples the relation, infers physical and semantic types, and chooses aggregations valid for each type.

- Numeric columns use the configured type/function map, commonly yielding sum, average, median, minimum, and maximum.
- Boolean columns are summed as 0/1 values, producing a count of true rows.
- Identifier-like columns contribute one general count instead of a redundant count for every identifier.
- Categorical columns get a distinct count. Selected values also get count, share, and presence features. Low-cardinality columns use all sampled values; high-cardinality columns use the most frequent values and an `other` bucket.
- Date and timestamp columns can receive min/max summaries.
- Text-looking columns can receive inexpensive SQL shape features when text features are enabled: character and word-length summaries plus count/share/presence for empty text, URLs, numbers, question marks, and exclamation marks.

For a time-series relation, the baseline also produces recency and volume:

- `seconds_since_last`;
- `num_events_{period}d`; and
- a safe short-window/long-window ratio for adjacent periods.

8.4.2 Example

Feature	Value	Meaning
`amount_sum`	60	Total amount over visible history
`amount_avg`	20	Mean amount per event
`status_nunique`	2	Two observed statuses
`status_invited_count`	2	Two invited events
`status_invited_share`	0.667	Two of three events were invited
`seconds_since_last`	86,400	Latest event was one day before cutoff
`num_events_7d`	2	Two events occurred in the 7-day window
`d1v7_change`	0.5	One 1-day event divided by two 7-day events

8.4.3 When to use it

Use `base` on nearly every reduced SQL child relation. It establishes a strong generic relational baseline and is inexpensive relative to combinatorial window families.

In the current SQL planner, conservative type-aware rollups form the baseline inside automatic feature synthesis and other family checks add operations around them. Omitting the literal string `base` is therefore not a reliable way to suppress every ordinary rollup.

8.5 `semantic`: explicitly supplied domain meaning

8.5.1 What it does

Automation status: custom/declarative; no domain semantics are inferred.

`semantic` lets the caller name domain concepts with SQL expressions. It does not guess that a column named `status` means success or that an amount of 100 is high value. The caller supplies those meanings:

```
annotation_expressions={
    "is_invited": "{status} = 'invited'",
    "is_high_value": "{amount} >= 25",
    "weighted_amount": ("value", "{amount} * {quantity}"),
}
```

A plain expression is a predicate compiled to a numeric 0/1 column. A `("value", expression)` annotation preserves the expression's numeric value. Column placeholders resolve against the node's actual, possibly prefixed, columns.

8.5.2 Example

For `is_invited = status = 'invited'`, the rows are annotated `1, 0, 1`. Ordinary numeric reduction can then produce:

Derived feature	Value
`is_invited_sum`	2
`is_invited_avg`	0.667
`is_invited_max`	1

If `conditional` is also enabled, the predicate can be measured inside every lookback window. If `temporal` is enabled, a numeric value annotation can receive windowed sum/average/min/max features.

8.5.3 When to use it

Use `semantic` when a stable domain rule is known and worth making explicit: a successful outcome, a severe incident, a premium transaction, a completed workflow, or a domain-specific amount. Such rules are configuration and must be disclosed when evaluating how generic a benchmark method is.

Adding `semantic` without `annotation_expressions` does not invent domain features. Generic auto-annotation is a separate bounded heuristic based on schema and sampled values; its outputs should be described as automatically annotated generic features, not inferred domain semantics.

8.6 `conditional`: composition within time windows

8.6.1 What it does

`conditional` answers *what kind of activity occurred?* It selects conditions from explicit predicate annotations and sampled categorical values. Predicate annotations are prioritized. Free-form text, collections, dates, identifiers, and the reduction key are excluded.

For every selected condition and period it emits:

- `count`: rows satisfying the condition;
- `share`: condition count divided by all rows in the window;
- `any`: whether the condition appeared at least once; and
- `change`: short-window condition count divided by the next longer-window condition count.

8.6.2 Example

For `status = 'invited'`:

Feature	Value	Explanation
`invited_count_7d`	1	Only the Jan 9 row is invited in 7 days
`invited_share_7d`	0.5	One of two 7-day events is invited
`invited_any_7d`	1	At least one invited event exists
`invited_d7v30_change`	0.5	One 7-day invite divided by two 30-day invites

8.6.3 When to use it and cost

Use it for status mix, action type, channel, outcome, category, or a sparse event where total volume is too coarse. With S selected conditions and P periods, the family adds approximately $S(4P-1)$ columns at one node. Bound the condition and category budgets, especially on deep graphs.

8.7 `temporal`: numeric magnitude by lookback window

8.7.1 What it does

`temporal` applies each configured lookback window to numeric measurements. Explicit numeric value annotations are selected first, followed by generic numeric columns, up to the family column budget. Identifiers, the reduction key, and the date key are excluded.

For every selected number and period it emits windowed sum, average, minimum, and maximum. SQL conditional aggregates turn values outside the window into null inputs rather than allowing them to leak into the aggregate.

8.7.2 Example

Window	Sum	Average	Minimum	Maximum
1 day	30	30	30	30
7 days	50	25	20	30
30 days	60	20	10	30

Two entities can have the same number of events but very different spending, duration, quantity, score, or severity. This family preserves that difference.

8.7.3 When to use it and cost

Use it on time-series relations with meaningful numeric measurements. With N selected numeric columns and P periods, it adds approximately $4NP$ columns at one node.

8.8 `sequence`: cadence, concentration, and active span

8.8.1 What it does

`sequence` describes the timing shape of activity without exposing an ordered event list. For every period it computes:

- `activity_rate_{period}d`: events in the window divided by window length;
- `activity_share_{period}d`: events in the window divided by lifetime events; and
- for adjacent windows, a short-window/long-window burst ratio.

Supported SQL dialects also receive `active_span_seconds`, the interval from first visible event to last visible event, and `activities_per_active_day`.

8.8.2 Example

Feature	Value	Meaning
`activity_rate_7d`	0.286	2 events / 7 days
`activity_share_7d`	0.667	2 recent events / 3 lifetime events
`activity_burst_1v7`	0.5	1 event / 2 events
`active_span_seconds`	691,200	Eight days between Jan 1 and Jan 9
`activities_per_active_day`	0.333	3 events / (8 + 1) days

8.8.3 When to use it and cost

Use it for churn, repeat behavior, engagement, burst detection, or time-to-event tasks. With P periods, it adds roughly $3P + 1$ columns when active-span arithmetic is supported.

8.9 `episode`: rows versus distinct records

8.9.1 What it does

`episode` counts rows and, when a single primary key is available, distinct primary keys. On time-series relations it repeats both counts inside every lookback window. This distinction matters after a many-to-many join, where one logical record can appear in several rows.

8.9.2 Example

Imagine a joined order/product relation:

order_id	product
101	A
101	B
102	C

Feature	Value	Interpretation
`num_episodes`	3	Three rows after the join
`num_unique_episodes`	2	Two distinct orders

The gap reveals multiplicity that a plain row count hides. Composite primary keys may require an explicit distinct-record policy.

8.9.3 When to use it and cost

Use it when join expansion, repeated records, or the distinction between line items and business events matters. With a usable single-column primary key, the family adds two lifetime columns and two per period.

8.10 `context`: peer-relative row values

8.10.1 What it does

`context` compares a row with a caller-defined peer group *before* reduction. The caller must provide context keys; Kurve RSC does not guess whether `race_id`, `merchant_id`, `category`, or `event_id` is the meaningful group.

For each resolved key it adds peer-group row count and, for selected numeric columns, the signed difference `value - peer_group_average(value)`. These row-level values then flow through ordinary reduction, so a parent receives summaries of its children's relative standing.

8.10.2 Example

Three drivers finish one race in positions 1, 4, and 2. Their race average is 2.333.

Driver position	Context size	Position minus race average
1	3	-1.333
4	3	1.667
2	3	-0.333

The feature is signed, not inherently good or bad. Interpretation depends on the measured quantity.

8.10.3 When to use it

Use it only when the peer group has a defensible relational or business meaning. Context keys and numeric candidates must remain bounded.

8.11 How the families compose

The families are additive and often most useful in pairs:

Combination	What it captures
`base` + `temporal`	Lifetime magnitude plus recent numeric magnitude
`semantic` + `conditional`	Domain-specific event mix in each window
`semantic` + `temporal`	Domain-specific numeric value by window
`base` + `sequence`	Overall volume/recency plus cadence and burstiness
`conditional` + `episode`	Condition rates with explicit row/distinct-record denominators
`context` + `base`	Peer-relative row signals summarized to parent grain

Text is a baseline capability rather than an eighth family. It measures shape and simple patterns; it does not tokenize text, create embeddings, or call a language model.

Family composition multiplies with depth. Enabling `temporal` on a depth-two child can create windowed features there, which may then be summarized again at depth one before reaching the root. This can be powerful, but it makes feature width and lineage important observability concerns.

8.12 Configuration example

This abbreviated configuration enables all non-context families for an event table. The graph still needs automatic features and SQL execution.

```
import datetime
import duckdb

from graphreduce.enum import ComputeLayerEnum, PeriodUnit
from graphreduce.graph_reduce import GraphReduce
from graphreduce.node import DuckdbNode

connection = duckdb.connect()

users = DuckdbNode(
    fpath="users",
    pk="user_id",
    prefix="usr",
    columns=["user_id"],
)

events = DuckdbNode(
    fpath="events",
    pk="event_id",
    prefix="evt",
    date_key="event_time",
    columns=["event_id", "user_id", "event_time", "status", "amount"],
    feature_families=(
        "base",
        "semantic",
        "conditional",
        "temporal",
        "sequence",
        "episode",
    ),
    annotation_expressions={
        "is_invited": "{status} = 'invited'",
        "is_high_value": "{amount} >= 25",
    },
    ts_periods=(1, 7, 30),
    feature_family_max_columns=4,
    categorical_top_k=5,
)

graph = GraphReduce(
    name="user_features_at_cutoff",
    parent_node=users,
    compute_layer=ComputeLayerEnum.duckdb,
    sql_client=connection,
    cut_date=datetime.datetime(2024, 1, 10),
    compute_period_val=30,
    compute_period_unit=PeriodUnit.day,
    auto_features=True,
    auto_labels=False,
    auto_feature_hops_back=2,
    auto_feature_hops_front=0,
)

graph.add_node(users)
graph.add_node(events)
graph.add_entity_edge(
    parent_node=users,
```

```
    relation_node=events,
    parent_key="user_id",
    relation_key="user_id",
    reduce=True,
)
graph.do_transformations_sql()
```

Kurve RSC places this graph construction inside the cutoff loop. The cutoff, eligible entity set, and labels change for each F_i; the schema, depth, and family policy remain fixed unless the experiment explicitly studies one of those controls.

9 Time windows and feature budgets

9.1 Window semantics

Correct temporal features depend on:

- a valid date key on every time-varying node;
- the cutoff, which supplies the point-in-time reference;
- the compute period, which bounds historical input rows; and
- the list of day windows used by `base`, `conditional`, `temporal`, `sequence`, and `episode`.

The common default windows are 1, 3, 4, 7, 14, 30, 60, 90, 180, 365, and 730 days. Empty periods disable rolling features. A window longer than the compute horizon cannot recover older rows because they have already been filtered.

9.2 Feature-family controls

Setting	Controls
`feature_families`	Requested named families; unknown names fail fast
`ts_periods`	Number and length of lookback windows
`feature_family_max_columns`	Numeric candidates and selected conditions
`categorical_cardinality_threshold`	All sampled categories versus a bounded subset
`categorical_top_k`	Size of the high-cardinality category subset
`annotation_expressions`	Explicit semantic predicates and values
`auto_annotate_features`	Generic bounded annotations inferred from sampled data
`auto_text_features`	Baseline text-shape and pattern features
`context_keys`	Explicit peer groups for `context`

Approximate extra columns at one relation help with planning:

Family	Approximate count
Baseline time features	$2P$ (recency, P counts, $P-1$ ratios)
`conditional`	$S(4P-1)$
`temporal`	$4NP$
`sequence`	$3P+1$ when span features are supported
`episode`	$2+2P$ with one usable primary key; otherwise $1+P$
`context`	Per peer key, one group-size value plus up to N numeric deltas before reduction

Here P is period count, S selected conditions, and N selected numeric columns. These estimates apply at one node and hop. Multiple tables, deeper propagation, and downstream re-aggregation can multiply the final width.

10 System budgeting and experimental design

10.1 The main cost surface

A useful first-order description of work is

$$\text{work} \approx K \times \text{GraphCost}(V_d, E_d, \mathcal{A}, P),$$

where V_d and E_d are the tables and edges included by depth, $\mathcal{A}$ is the family policy, and P is the temporal-window count. This is not a runtime guarantee: join cardinality, sampled categories, SQL optimizer choices, and disk behavior can dominate. It is a useful design model because it keeps the row axis (K) separate from the relational-width axis (d and families).

10.2 Recommended staged rollout

1. Start with $K = 1$, shallow depth, and `base`; verify entity grain, keys, and exact time boundaries.
2. Increase d one hop at a time and inspect which tables and edges become reachable.
3. Expand K after one frame's width and memory profile are understood.
4. Add `temporal` on numeric event tables and `sequence` where cadence matters.
5. Add bounded `conditional` and `episode` policies where composition or multiplicity matters.
6. Add explicit `semantic` and `context` logic only when its domain meaning is defensible and disclosed.
7. Compare joint and incremental fitting under the same K, d, and feature schema.

10.3 Minimum run manifest

A reproducible result should record:

- dataset/task identity and library versions;
- exact training, validation, and test cutoff lists;
- K, worker count, and frame spill policy;
- root table, included table/edge set, and directional depth parameters;
- compute horizon, timestamp boundary convention, and window list;
- enabled families and every family budget;
- every semantic annotation expression and context key, plus whether generic automatic annotation was enabled;
- final ordered feature schema or its stable digest;
- estimator backend, candidate configurations, selected configuration, and random seed; and
- official metric and prediction-key validation result.

11 Common pitfalls

Calling worker count frame count. Fifteen workers can build fifty frames. The former controls concurrency; K controls the number of training views.

Horizontally joining cutoff frames. Cutoffs are independent observations and should be row-unioned after schema alignment. Joining them by entity can mix information from different prediction times.

Assuming depth zero means no relational model. It still creates a point-in-time root frame, but it excludes relational tables. Depth and family selection should be reported together.

Claiming total schema coverage with a shallow depth. Metadata discovery can register every table while hop limits prevent distant signals from reaching the root. Verify the actual included subgraph and final lineage.

Ignoring cycles. A database can contain more foreign-key edges than a cycle-free traversal retains. Report the edge policy, not only table count.

Generating every family everywhere. Width compounds across graph hops. Place specialized families where their inputs and intended signal are clear.

Calling automatic annotations semantic inference. Generic annotation heuristics do not know the task's domain meaning. A semantic feature is a caller-supplied rule and should be reported as custom declarative configuration.

Assuming a window expands history. A 365-day period cannot see a year of data if the compute horizon is only 90 days.

Using a category as free-form text, or text as a category. Inference is sample-dependent. Inspect representative samples and bound high-cardinality expansion.

Confusing rows with business events. Many-to-many joins can duplicate a primary key. Compare row and distinct-record counts when multiplicity matters.

Choosing a plausible but wrong context key. Peer-relative features can look reasonable even when the peer group has no causal or business meaning.

Selecting on test performance. Depth, frame count, families, model configuration, clipping, and blending must be fixed without test-label input.

12 Related work and positioning

12.1 Automated relational feature synthesis

Deep Feature Synthesis (DFS) established an influential program for composing transformation and aggregation primitives across normalized relational data [5]. Kurve RSC shares its goal of replacing repeated hand-written feature SQL with reusable relational operations. Kurve RSC emphasizes a temporal extension of that problem: the compute graph is instantiated at K cutoffs, every output has an explicit prediction-time contract, graph depth is an experimental control, and the resulting frame parts participate in one model lifecycle.

GraphReduce [1] is the direct execution substrate for Kurve RSC. It represents tables and relationships as a compute graph and provides the operation ordering, SQL synthesis, reductions, and feature propagation on which the Kurve RSC cutoff loop is built.

12.2 Relational deep learning and RelBench

Relational Deep Learning (RDL), developed by Stanford and Kumo.ai researchers, models database rows as nodes in a temporal heterogeneous graph and learns by message passing across primary–foreign key links [3]. RelBench supplies the open task, split, and evaluation infrastructure for comparing approaches to relational prediction [2]. Kurve RSC uses that same relational structure but compresses it into point-in-time tabular frames before learning, making it a complementary representation strategy rather than a graph-neural architecture.

Kumo.ai's more recent KumoRFM-2 work moves further toward a pretrained model that natively consumes connected relational tables and supports in-context learning and fine-tuning [8]. Kurve RSC draws a different system boundary: its relational compiler emits a stable dataframe and deliberately permits the downstream learner to be replaced.

12.3 Tabular learners and foundation models

CatBoost remains a strong conventional reference for heterogeneous tabular data [4]. TabPFN demonstrates a different paradigm: a transformer pretrained over many synthetic tabular tasks can condition on a new table in context [6]. Prior Labs' TabPFN-3 report extends that line toward larger tables and broader structured-data settings [7]. These systems make Kurve RSC's learner-neutral boundary particularly useful: the same relational frames can feed a fitted tree ensemble today and a pretrained tabular learner without changing the upstream time and graph semantics.

12.4 Embedded analytical execution

Kurve RSC's reference SQL path uses DuckDB, an in-process analytical database designed for embedded OLAP workloads [9]. This enables graph-generated SQL, temporary relations, and dataframe interchange to coexist in one task process. DuckDB is an execution choice rather than a requirement of the Kurve RSC frame abstraction.

13 Implementation boundaries

13.1 What Kurve RSC owns

Kurve RSC is the end-to-end benchmark implementation. It owns:

- adapters from task/database metadata into compute graphs;
- selection and ordering of cutoff frames;
- graph depth and feature-family policy;
- point-in-time entity filtering and task-frame joins;
- concurrent construction and disk-backed frame storage;
- feature alignment and model-input normalization;
- validation-based estimator selection and joint/incremental fitting;
- official evaluation, reporting, and submission integrity checks.

13.2 What GraphReduce owns

GraphReduce is a separate open-source project and the relational execution engine used by Kurve RSC. It owns the reusable graph abstractions and execution machinery for table nodes, relationship edges, operation ordering, SQL generation, temporal filtering, reduction, and automatic feature propagation. The project source and documentation are available at github.com/wesmadrigal/graphreduce, with packaged releases on PyPI [1].

This distinction matters for attribution and reproducibility: Kurve RSC is not a rename of GraphReduce. It is a framework built **with** GraphReduce, adding the repeated cutoff-frame lifecycle and benchmark/model protocol described in this report.

13.3 What the downstream learner owns

The learner owns statistical adaptation from the aligned dataframe to the target: supervised fitting, in-context conditioning, optional gradient fine-tuning, and prediction. CatBoost and TabPFN exercise different versions of that contract, but neither changes which database rows were visible at a cutoff or how relational signal reached the entity grain.

13.4 What the benchmark owns

RelBench [2,3] supplies realistic relational databases, primary–foreign key metadata, task definitions, official temporal splits, task tables, and unified evaluators. Kurve RSC does not redefine those labels or metrics. Its role is to construct and train on a different representation of the same official task.

14 Limitations and open engineering questions

Kurve RSC deliberately makes its largest trade-offs explicit rather than pretending they disappear:

- A larger K increases temporal coverage but may overweight entities that appear at many cutoffs unless the task schedule is interpreted carefully.
- A larger d increases relational coverage but can produce very wide SQL plans and memory-heavy intermediates.
- Sample-driven categorical and type inference can vary when late cutoffs expose new values; schema freezing and deterministic samples are important.
- Incremental CatBoost is memory efficient but is not mathematically identical to a single joint fit on the concatenated matrix.
- Tabular foundation models have context, feature-count, memory, and licensing constraints that can change the feasible K, d, and family budget.
- Cyclic schemas require an explicit path policy. A spanning tree covers all reachable tables but not every possible relationship path.
- Semantic and context features can encode useful knowledge, but they are no longer purely schema-derived and must be disclosed as such.
- Generic relational coverage does not guarantee useful signal. Validation remains necessary to select budgets without consulting test labels.

Future work includes automatic resource estimation before execution, stable feature-lineage manifests, cost-aware selection of K and d, schema-cycle strategies that preserve additional paths without duplicate explosion, and cross-task policies that allocate specialized families by type rather than by target identity.

15 Conclusion

Kurve RSC treats model-ready data as the output of a repeated relational computation, not as one manually assembled table. At cutoff c_i, a depth-bounded GraphReduce instantiation converts visible multi-table history into one entity-grain frame. Across K cutoffs, those frames form a unified longitudinal training relation. Any compatible downstream learner can then be fit, conditioned, or fine-tuned on that representation under official task splits.

The architecture has three clear axes:

1. K controls how many point-in-time training views are generated;
2. d controls how far relational signal can propagate; and
3. Kurve RSC Feature Families control what signal survives each reduction.

Keeping these axes separate makes the system easier to reason about, scale, audit, and reproduce. It also makes the central claim precise: Kurve RSC's contribution is the point-in-time compression of relational history into a strong tabular representation, while GraphReduce supplies the general-purpose relational execution substrate and RelBench supplies the official evaluation contract.

16 References

1. W. Madrigal. **GraphReduce: relational feature engineering through graphs of tables, keys, and compute operations.** Open-source software project. GitHub, documentation, and PyPI.
2. J. Robinson, R. Ranjan, W. Hu, K. Huang, J. Han, A. Dobles, M. Fey, J. E. Lenssen, Y. Yuan, Z. Zhang, X. He, and J. Leskovec. **RelBench: A Benchmark for Deep Learning on Relational Databases.** NeurIPS Datasets and Benchmarks, 2024. arXiv:2407.20060.
3. M. Fey, W. Hu, K. Huang, J. E. Lenssen, R. Ranjan, J. Robinson, R. Ying, J. You, and J. Leskovec. **Relational Deep Learning: Graph Representation Learning on Relational Databases.** ICML, 2024. PMLR and arXiv:2312.04615.
4. L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. **CatBoost: Unbiased Boosting with Categorical Features.** NeurIPS, 2018. Paper and project references.
5. J. M. Kanter and K. Veeramachaneni. **Deep Feature Synthesis: Towards Automating Data Science Endeavors.** IEEE International Conference on Data Science and Advanced Analytics, 2015. doi:10.1109/DSAA.2015.7344858.
6. N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. **Accurate Predictions on Small Data with a Tabular Foundation Model.** *Nature* 637, 319–326, 2025. doi:10.1038/s41586-024-08328-6.
7. Prior Labs Team. **TabPFN-3: Technical Report.** May 12, 2026. Prior Labs technical report.
8. V. Hudovernik, F. López, V. Kocijan, A. Nitta, J. E. Lenssen, J. Leskovec, and M. Fey. **KumoRFM-2: Scaling Foundation Models for Relational Learning.**

   2026. arXiv:2604.12596.
9. M. Raasveldt and H. Mühleisen. **DuckDB: An Embeddable Analytical Database.** ACM SIGMOD, 2019. doi:10.1145/3299869.3320212.